Assignment 10: File System (fs)

CSci 530: Operating Systems

Objectives

In this lab you will add large files and symbolic links ot the xv6 file system.

Before writing code, make sure to read the following parts from the xv6 book:

- Chapter 10: File system

Description

Have a look at the Getting Started instructions for information on how to set up your computer to work on and run these assignments.

Note: Remember that assignments are submitted to GitHub classrooms by creating a commit and pushing it back to your assignment repository. You are required to make a commit for at least each task as you complete it (though you can make more than one commit for a task if needed). In general, from the command line, you can create and push a commit by doing something like:

```
$ git commit -am "My solution for Assignment 10 Task 01 large files"
$ git push
```

Assignment Tasks

Task 01: Large Files (moderate)

In this assignment you'll increase the maximum size of an xv6 file. Currently xv6 files are limited to 268 blocks, or 268*BSIZE bytes (`BSIZE` is 1024 in xv6). This limit comes from the fact that an xv6 inode contains 12 "direct" block numbers and one "singly-indirect" block number, which refers to a block that holds up to 256 more block numbers, for a total of 12+256=268 blocks.

The bigfile command creates the longest file it can, and reports that size:

```
$ bigfile
..
wrote 268 blocks
bigfile: file is too small
$
```

The test fails because bigfile expects to be able to create a file with 65803 blocks, but unmodified xv6 limits files to 268 blocks.

You'll change the xv6 file system code to support a "doubly-indirect" block in each inode, containing 256 addresses of singly-indirect blocks, each of which can contain up to 256 addresses of data blocks. The result will be that a file will be able to consist of up to 65803 blocks, or 256*256+256+11 blocks (11 instead of 12, because we will sacrifice one of the direct block numbers for the double-indirect block).

Preliminaries

The `mkfs` program, which you can run using `make fs.img`, creates the xv6 file system disk image and determines how many total blocks the file system has; this size is controlled by `FSSIZE` in `kernel/param.h`. You'll see that `FSSIZE` in the repository for this lab is set to 200,000 blocks. You should see the following output from mkfs/mkfs in the make output:

```
nmeta 71 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 25) blocks 199929 total 200000
```

This line describes the file system that mkfs/mkfs built: it has 71 meta-data blocks (blocks used to describe the file system) and 199,929 data blocks, totaling 200,000 blocks. Note that make qemu builds a new fs.img, and saves the old one in fs.img.bk. If you want to run xv6 with the existing fs.img instead of building a new one, run make qemu-fs.

What to look at

The format of an on-disk inode is defined by `struct dinode` in `fs.h`. You're particularly interested in `NDIRECT`, `NINDIRECT`, `MAXFILE`, and the `addrs[]` element of `struct dinode`. Look at Figure 10.3 in the xv6 text for a diagram of the standard xv6 inode. The code that finds a file's data on disk is in `bmap()` in `fs.c`. Have a look at it and make sure you understand what it's doing. `bmap()` is called both when reading and writing a file. When writing, `bmap()` allocates new blocks as needed to hold file content, as well as allocating an indirect block if needed to hold block addresses.

`bmap()` deals with two kinds of block numbers. The `bn` argument is a "logical block number" – a block number within the file, relative to the start of the file. The block numbers in `ip->addrs[]`, and the argument to `bread()`, are disk block numbers. You can view `bmap()` as mapping a file's logical block numbers into disk block numbers.

Your Job

Modify `bmap()` so that it implements a doubly-indirect block, in addition to direct blocks and a singly-indirect block. You'll have to have only 11 direct blocks, rather than 12, to make room for your new doubly-indirect block; you're not allowed to change the size of an on-disk inode. The first 11 elements of `ip->addrs[]` should be direct blocks; the 12th should be a singly-indirect block (just like the current one); the 13th should be your new doubly-indirect block. You are done with this exercise when `bigfile` writes 65803 blocks and `usertests -q` runs successfully:

```
$ bigfile
..........................................................................................................................
wrote 65803 blocks
done; ok
$ usertests -q
...
ALL TESTS PASSED
$
```

`bigfile` will take at least a minute and a half to run.

Hints

- Make sure you understand `bmap()`. Write out a diagram of the relationships between `ip->addrs[]`, the indirect block, the doubly-indirect block and the singly-indirect blocks it points to, and data blocks. Make sure you understand why adding a doubly-indirect block increases the maximum file size by 256*256 blocks (really -1, since you have to decrease the number of direct blocks by one).
- Think about how you'll index the doubly-indirect block, and the indirect blocks it points to, with the logical block number.
- If you change the definition of `NDIRECT`, you'll probably have to change the declaration of `addrs[]` in `struct inode` in `file.h`. Make sure that `struct inode` and `struct dinode` have the same number of elements in their `addrs[]` arrays.
- If you change the definition of `NDIRECT`, make sure to create a new `fs.img`, since `mkfs` uses `NDIRECT` to build the file system.
- If your file system gets into a bad state, perhaps by crashing, delete `fs.img` (do this from Unix, not xv6). `make` will build a new clean file system image for you.

- Don't forget to `brelse()` each block that you `bread()`.
- You should allocate indirect blocks and doubly-indirect blocks only as needed, like the original `bmap()`.
- Make sure `itrunc` frees all blocks of a file, including double-indirect blocks.
- `usertests` takes longer to run than in previous labs because for this lab `FSSIZE` is larger and big files are larger.

Task 02: Symbolic Links (moderate)

In this exercise you will add symbolic links to xv6. Symbolic links (or soft links) refer to a linked file or directory by pathname; when a symbolic link is opened, the kernel looks up the linked-to name. Symbolic links resemble hard links, but hard links are restricted to pointing to files on the same disk, cannot refer to directories, and are tied to a specific target i-node rather than (as with symbolic links) referring to whatever happens at the moment to be at the target name, if anything. Implementing this system call is a good exercise to understand how pathname lookup works.

You do not have to handle symbolic links to directories for this lab; the only system call that needs to know how to follow symbolic links is `open()`.

Your job

You will implement the `symlink(char *target, char *path)` system call, which creates a new symbolic link at path that refers to file named by target. For further information, see the man page symlink. To test, add `symlinktest` to the Makefile and run it. Your solution is complete when the tests produce the following output (including `usertests` succeeding).

```
$ symlinktest
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
$ usertests -q
...
ALL TESTS PASSED
$
```

Submit the Assignment

Time spent

Create a new file, `time.txt`, and put in a single integer, the number of hours you spent on the lab. Make sure you add and commit a final commit with this file as it is part of the autograder score.

Assignment submissions are handled by the GitHub classroom autograder. You should create at least 1 commit for each task as you do it, and push it to your GitHub classroom. You can check that the autograder tests are passing in GitHub classroom by looking at your submitted releases. The tests run in the GitHub classroom are the same ones that you can run locally by using `make grade` or `./grade-lab-lock test`.

- Please run `make grade` to ensure that your code passes all of the tests. The GitHub classroom autograder will use the same grading program to assign you an autograder evaluation score for your work.

Optional Challenge Exercises